MINDSTORMS Robot Inventor - Word Blocks Contents

Contents i
Word Block Types 6
Hat Blocks 6
Stack Blocks 6
C Blocks 6
Reporter Blocks 7
Boolean Blocks 7
Cap Blocks 7
Block Stack 8
Word Block Descriptions 9
Component Names 9
Parameters 9
Motor Blocks Category 10
Motor: (Hub Port▼) Run (Direction▼) For (Value) (Units▼) 10
Motor: (Hub Port▼) Go (Direction▼) To Position (Value) 10
Motor: (Hub Port▼) Start Motor (Direction▼) 10
Motor: (Hub Port▼) Stop Motor 10
Motor: (Hub Port▼) Set Speed To (Value) % 11
Motor: =(Hub Port▼) Position 11
Motor: =(Hub Port▼) Speed 11
Movement Blocks 11
Motor Pair: Move (Direction▼) For (Value) (Units▼) 11
Motor Pair: Move (Value Steering) For (Value) (Units▼) 11
Motor Pair: Start Moving (Value Steering) 12
Motor Pair: Stop Moving 12
Motor Pair: Set Movement Speed To (Value) % 12
Motor Pair: Set Movement Motors To (Hub Ports▼) 12
Motor Pair: Set 1 Motor Rotation To (Value) (Units▼) Moved 12
Light Blocks Category 13
Hub: Start Animation (Name▼) 13
Hub: Play Animation (Name▼) Until Done 13
Hub: Turn On (5x5 Pattern▼) For (Value) Seconds 13
Hub: Turn On (5x5 Pattern▼) 13
Hub: Write (String) 13
Hub: Turn Off Pixels 14
Hub: Set Pixel Brightness To (Value) % 14
Hub: Set Pixel At (X▼), (Y▼) To (Value) % 14
Hub: Rotate Orientation (Direction▼) 14
Hub: Set Orientation To (Direction▼) 14
Hub: Set Centre Button Light To (Colour▼) 14
Distance Sensor: (Hub Port▼) Light Up (Lights Pattern▼) 15

Sound Blocks 15
App: Play Sound (Name▼) Until Done 15
App: Start Sound (Name▼) 15
Hub: Play Beep (Tone Value) For (Value) Seconds 15
Hub: Start Playing Beep (Tone Value) 15
App & Hub: Stop All Sounds 15
App: Change (Pitch Or Pan▼) Effect By (Value) 16
App: Set (Pitch Or Pan▼) Effect To (Value) 16
App: Clear Sound Effects 16
App: Change Volume By (Value) 16
App: Set Volume To (Value) 16
App: =Volume 16
Event Blocks 17
When Program Starts 17
Colour Sensor: (Hub Port▼) When Colour Is (Colour▼) 17
Distance Sensor: (Hub Port▼) When (Closer/Further Than▼) (Value) (Units) 17
Hub: When (Hub Orientation▼) Is Up 18
Hub: When (Shaken/Tapped/Falling▼) 18
Hub: When (Left/Right▼) Button (Pressed/Released▼) 18
When <Condition> 18
When I Receive (Message▼) 19
Broadcast (Message▼) 19
Broadcast (Message▼) and Wait 19
When Timer > (Value) 19
When (Key Name▼) Key Pressed 19
Control Blocks 20
Wait (Value) Seconds 20
Wait Until <Condition> 20
Repeat (Value) [Block Stack] 20
Forever [Block Stack] 20
Repeat Until <Condition> [Block Stack] 20
If <Condition> Then [Block Stack] 21
If <Condition> Then [Block Stack] Else [Block Stack] 21
Stop Other Stacks 21
Stop (Stacks▼) 21
Sensor Blocks 22
Colour Sensor: =(Hub Port▼) Is Colour (Colour▼)? 22
Colour Sensor: =(Hub Port▼) Colour 22
Colour Sensor: =(Hub Port▼) Reflection (Compare▼) (Value) %? 22
Colour Sensor: =(Hub Port▼) Reflected Light 23
Distance Sensor: =(Hub Port▼) Is (Closer/Further Than▼) (Value) (Units▼)? 23
Distance Sensor: =(Hub Port▼) Distance In (Units▼) 23
Hub: =Is (Hub Orientation▼) Up? 23
Hub: =(Pitch/Roll/Yaw▼) Angle 23
Hub: Set Yaw Angle to 0 24
Hub: =Is (Left/Right▼) Button (Pressed/Released▼)? 24
App: =Key (Key Name▼) Pressed? 24

Hub: =Timer 24
Hub: Reset Timer....24
Hub: =Gesture 24
Hub: =Orientation 25
Operator Blocks....25
=Pick Random (Start Value) To (End Value) 25
=(Value1) Plus (Value2) 25
=(Value1) Minus (Value2)....25
=(Value1) Multiplied By (Value2) 25
=(Value2) Divided By (Value1) 25
=(Value1) Greater Than (Value2)....26
=(Value1) Less Than (Value2) 26
=(Value1) Equal To (Value2)....26
=<Boolean Block1> And <Boolean Block2> 26
=<Boolean Block1> Or <Boolean Block2> 26
=Not <Boolean Block> 26
=Is (Value1) In Between (Value2) And (Value3)?....26
=Join (String1) (String2) 26
=Letter (Value) Of (String) 27
=Length Of (String) 27
=(String) Contains (Character)? 27
=(Value1) Mod (Value2) 27
=Round (Value)....27
=(Math Function) Of (Value) 27
Controller Commands 28
App RC: When Joystick (J Name▼) Is (Position▼) 28
App RC: =Joystick (J Name▼) (Axis▼)....28
App RC: When D-Pad (D Name▼) (B Name▼) Button Is (Pressed/Released▼)....28
App RC: =Is D-Pad (D Name▼) (B Name▼) Button (Pressed/Released▼)?....28
App RC: When Slider (S Name▼) Is (Position▼) 28
App RC: =Slider (S Name▼) 29
App RC: When Button (B Name▼) Is (Pressed/Released▼) 29
App RC: =Is Button (B Name▼) (Pressed/Released▼)? 29
App RC: When Switch (S Name▼) Is Toggled (On/Off▼)....29
App RC: =Is Switch (S Name▼) (On/Off▼)? 29
Variable Blocks....30
Make A Variable 30
=(MyVariable)....30
Set (Variable▼) To(Value)....30
Change (Variable▼) By (Value)....30
Make A List....30
=(MyList) 30
Add (Item) To (List▼) 31
Delete (Index) Of (List▼)....31
Delete All Of (List▼)....31
Insert (Item) At (Index) Of (List▼)....31
Replace Item (Index) Of (List▼) With (Item) 31

=Item (Index) Of (List▼) 31
=Item # Of (Item) In (List▼) 31
=Length Of (List▼) 31
=(List▼) Contains (Item)? 32
My Block 32
Make A Block 32
MyBlock 32
More Motor Blocks 32
Motor: (Hub Port▼) Run For (Value) (Unit▼) At (Value) % Speed 32
Motor: (Hub Port▼) Start Motor At (Value) % Speed 32
Motor: (Hub Port▼) Go To Relative Position (Value) At (Value) % Speed 33
Motor: (Hub Port▼) Set Relative Position To (Value) 33
Motor: =(Hub Port▼) Degrees Counted 33
Motor: (Hub Port▼) Start Motor At (Value) % Power 33
Motor: =(Hub Port▼) Power 33
Motor: (Hub Port▼) Set Motors To (Brake/Hold/Float▼) At Stop 34
Motor: (Hub Port▼) Turn Stall Detection (On/Off▼) 34
Motor: (Hub Port▼) Was The Motor Interrupted? 34
Motor: (Hub Port▼) Set Acceleration To (Rate▼) 34
More Movement Blocks 35
Motor Pair: Move For (Value) (Units▼) At (L Value) (R Value) % Speed 35
Motor Pair: Start Moving At (L Value) (R Value) % Speed 35
Motor Pair: Move (Value Steering) For (Value) (Units▼) At (Value) % Speed 35
Motor Pair: Start Moving (Value Steering) At (Value) % Speed 35
Motor Pair: Start Moving At (L Value) (R Value) % Power 35
Motor Pair: Start Moving (Value Steering) At (Value) % Power 36
Motor Pair: Set Movement Motors To (Brake/Hold/Float▼) At Stop 36
Motor Pair: =Did Movement Get Interrupted? 36
Motor Pair: Set Movement Acceleration To (Rate▼) 36
Weather Blocks 37
App Weather: Set Location To (String) 37
App Weather: =Location 37
App Weather: =Temperature In (°C/°F▼) 37
App Weather: =Wind Speed In (Units▼) 37
App Weather: =Wind Direction 37
App Weather: =Is (Weather Condition▼)? 38
App Weather: =Precipitation In (Units ▼) 38
App Weather: =Air Pressure In hPa 38
App Weather: Set Forecast To Now 38
App Weather: Set Forecast To (Day ▼) At (Value) O'Clock 38
App Weather: Change Forecast By (Value) Hours 38
App Weather: =Forecast Time 39
Music Blocks 39
App Music: Play Drum (Name▼) For (Value) Beats 39
App Music: Rest For (Value) Beats 39
App Music: Play Note (Tone) For (Value) Beats 39
App Music: Set Instrument To (Name▼) 39

App Music: Set Tempo To (Value) 39
App Music: Change Tempo By (Value)....40
App Music: =Tempo 40
Gelo: Commands 40
Gelo: Walk (Direction▼) For (Value) Seconds 40
Gelo: Move For (Value) Seconds 40
Gelo: Set Speed To (Value) % 40
Gelo: Set Legs To (P Name▼) Position 40
Gelo: Start Moving 41
Gelo: Stop 41
M.V.P.: Commands....41
M.V.P.: Calibrate 41
M.V.P.: Remote Control With X (ValueX) And Y (ValueY) 41
Tricky: Commands 42
Tricky: Remote Control With X (ValueX) And Y (ValueY)....42
Charlie: Commands....42
Charlie: Calibrate....42
Charlie: Play Drums For (Value) Bars At Tempo (Value) 42
Charlie: Remote Control With X (ValueX) And Y (ValueY) 42
Blast: Commands....43
Blast: Calibrate....43
Blast: Toggle Action 43
Blast: Remote Control With X (ValueX) And Y (ValueY) 43
More Controller Commands 43
DualShock®4 RC: When (S Name▼) Stick Is (Position▼)....43
Xbox One® RC: When (S Name▼) Stick Is (Position▼)....43
DualShock®4 RC: =(S Name▼) Stick (Axis▼) 44
Xbox One® RC: =(S Name▼) Stick (Axis▼) 44
DualShock®4 RC: When D-Pad (B Name▼) Button Is (Pressed/Released▼) 44
Xbox One® RC: When D-Pad (B Name▼) Button Is (Pressed/Released▼) 44
DualShock®4 RC: =Is D-Pad (B Name▼) Button (Pressed/Released▼)? 44
Xbox One® RC: =Is D-Pad (B Name▼) Button (Pressed/Released▼)? 44
DualShock®4 RC: When (B Name▼) Is (Pressed/Released▼)....45
Xbox One® RC: When (B Name▼) Is (Pressed/Released▼)....45
DualShock®4 RC: =Is (B Name▼) (Pressed/Released▼)?....45
Xbox One® RC: =Is (B Name▼) (Pressed/Released▼)?....45
DualShock®4 RC: =(T Name▼) Pressure....45
Xbox One® RC: =(T Name▼) Pressure....45
Index....46

Word Block Types

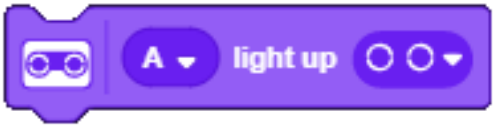
The Scratch programming language is made up of different types of blocks, each represented by a different shape.

Hat Blocks

Hat Blocks are used to start a program and start a stack of blocks when a particular event occurs. They have a rounded top so that other blocks can only be attached under them.

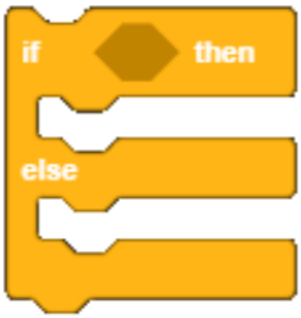
Stack Blocks

Stack Blocks perform the main commands in a program. They are the blocks that can make the motors move and the lights light up!



C Blocks

C Blocks are C-shaped blocks. They are placed between the beginning and the end of the loop or check whether a condition is “true.” All the C Blocks can be found in the *Control* category.



Reporter Blocks

Reporter Blocks hold values, which can be a number or a string. Among other things, they can hold a sensor reading or store the value of a variable.

Boolean Blocks

Boolean Blocks are conditions that can either be true or false. They are used together with C Blocks to form the logic of a program.

Cap Blocks

Cap Blocks are used to end scripts. They are shaped with a notch at the top and a flat bottom so that you cannot place any blocks below them. There are two Cap Blocks, both of which can be found in the *Control* category.

Block Stack

A *block stack* is a number of blocks that have been put together.

Word Block Descriptions

The following conventions have been used in the **Names** of each Block and its **Parameters**:

Component Names

The **Name** of the Lego™ Mindstorms™ component to which a Word Block applies. If there is no name, the Word Block is not associated with any component. For example, a **C Block**.

App:	The **Device** you are using to program your robots – a Smartphone, Tablet, Mac or PC. In addition to programming, you can use it to:
App RC:	Design your own on-screen **Remote Controller.**
App Music:	Write and Play **Musical Sounds** through your **Device's Speaker.**
App Weather:	Use **Weather Forecast** data when your **Device** is online.
Hub:	The **Mindstorms™ Hub** itself.
Colour Sensor:	Identified by the **Hub Port** into which it has been plugged.
Distance Sensor:	Identified by the **Hub Port** into which it has been plugged.
Motor:	Identified by the **Hub Port** into which it has been plugged.
Motor Pair:	A special case where **Two Motors** are controlled from **Two Hub Ports** as if they are one component. Used for Robots built on **Driving Bases**.
DualShock®4 RC:	PS4 **Remote Controller**.
Xbox One® RC:	Xbox One **Remote Controller**.

Parameters

Where a Word Block requires some information to do its work, various **Parameter** formats are used. In general, **Parameters** enclosed in **( )** can be typed in or be derived from an embedded **Reporter Block** . Those enclosed in **< >** **must** be derived from an embedded **Boolean Block**.

(Value)	A **Number**, either typed into the Word Block or derived from an embedded **Reporter Block**. These are mostly Integers (whole numbers) but can be Decimal in some Blocks.
(String)	A **Text String**, either typed into the Word Block or derived from an embedded **Reporter Block**.
(Character)	A **Single-Character Text String**, either typed into the Word Block or derived from an embedded **Reporter Block**.
(Name▾)	A Value derived from a Drop Down Selector within a Word Block. E.g. **(Hub Port▾)** is a Drop Down Selector used to select the letter representing a port on the Mindstorms™ Hub.
<Condition>	A Boolean (True or False) value which **must** be derived from an embedded **Boolean Block**.
[Block Stack]	A series of Word Blocks embedded into a **C Block**.

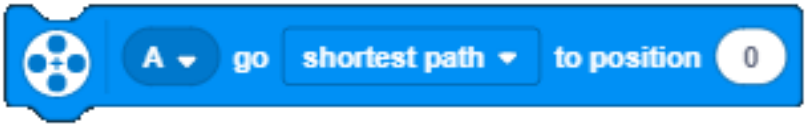
Motor Blocks Category

Motor Blocks either make the motors run or get information from the motors. The *Motor Blocks* category contains the most common Motor Blocks.

Motor: (Hub Port▾) Run (Direction▾) For (Value) (Units▾)

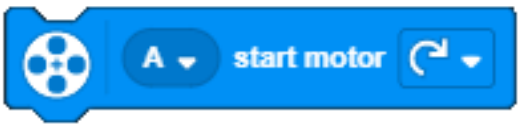
This block will run one or more motors clockwise or counterclockwise for a specified number of rotations, seconds, or degrees. The motor speed is set by the Set Speed Block. The default speed is 75%.
"Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: (Hub Port▾) Go (Direction▾) To Position (Value)



This block sets one or more motors to a specified position. The motor can be set to run clockwise, counterclockwise, or to take the shortest path to the specified position. The position ranges from 0 to 359 degrees. The motor speed is set by the Set Speed Block. The default speed is 75%.
"Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: (Hub Port▾) Start Motor (Direction▾)



This block will run one or more motors clockwise or counterclockwise forever. The motor speed is set by the Set Speed Block. The default speed is 75%."Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: (Hub Port▾) Stop Motor

This block stops one or more motors from running. The motor will brake so that it quickly comes to a complete stop. The motor will not hold its position once it has stopped.

Motor: (Hub Port▼) Set Speed To (Value) %

This block sets the speed of one or more motors. The speed range is -100 to 100. Negative values will reverse the direction of the motor. If the speed has not been specified, the default value is 75%.

Motor: =(Hub Port▼) Position

This block reports the current position of a motor. The position is given in degrees from 0 to 359.

Motor: =(Hub Port▼) Speed

This block reports the current speed of a motor. The value given is the motor's actual speed, not the speed set by the Set Speed Block.

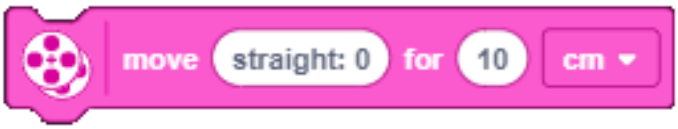
Movement Blocks

Movement Blocks enable you to run two motors in a synchronized motion. They are primarily used to move Driving Bases around. Only motors of the same type (e.g., two Medium Motors) can be synchronized.

Motor Pair: Move (Direction▼) For (Value) (Units▼)

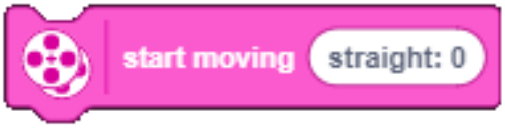
This block moves a Driving Base either forward or backward for a specified number of centimetres, inches, seconds, degrees, or rotations. The distance moved in centimetres and inches depends on how your Driving Base is built. Use the Set Distance to Motor Rotations Block to calibrate your Driving Base.

Motor Pair: Move (Value Steering) For (Value) (Units▼)



This block moves a Driving Base forward for a certain duration with the possibility of steering. Higher steering values (i.e., +99 and -99) will make the arc path of the Driving Base sharper. Use a value of"0" to drive in a straight line. Using the values 100 and -100 will make the Driving Base pivot on itself.

Motor Pair: Start Moving (Value Steering)



This block starts moving a Driving Base forward with the possibility of steering forever. Higher steering values (i.e., +99 and -99) will make the arc path of the Driving Base sharper. Use a value of"0" to drive in a straight line. Using the values 100 and -100 will make the Driving Base pivot on itself.

Motor Pair: Stop Moving

This block stops all movement of the Driving Base by turning off the motors.

Motor Pair: Set Movement Speed To (Value) %

This block sets the speed of a moving Driving Base. The speed range is -100 to 100. Negative values change the direction of the movement. The default value is 75%.

Motor Pair: Set Movement Motors To (Hub Ports▼)

This block defines the Ports to which the 2 driving motors are connected.

Motor Pair: Set 1 Motor Rotation To (Value) (Units▼) Moved

This block calibrates the distance of a Driving Base so that the distance unit (i.e., centimetres/inches) specified in the Movement Blocks will be accurate.

Light Blocks Category

Light Blocks enable you to turn the lights of different elements (e.g., 5x5 Light Matrix of the Hub, Distance Sensor lights) on and off.

Hub: Start Animation (Name▼)

This block plays a selected animation on the 5x5 Light Matrix and immediately plays the next block in the programming stack. You can add more animation to the project using the Add Animation Button.

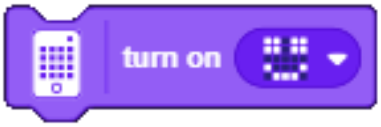
Hub: Play Animation (Name▼) Until Done

This block plays a selected animation on the 5x5 Light Matrix and pauses the programming stack until the animation is finished. You can add more animation to the project using the Add Animation Button.

Hub: Turn On (5x5 Pattern▼) For (Value) Seconds

This block creates a pattern and makes it light up on the 5x5 Light Matrix for a specified amount of time. The block will turn the pixels off when the time is up.

Hub: Turn On (5x5 Pattern▼)

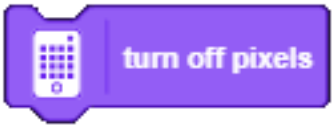


This block creates a pattern and makes it light up on the 5x5 Light Matrix. The pattern stays lit up until the Light Matrix is told to do something else or the program is stopped.

Hub: Write (String)

This block displays a text string on the 5x5 Light Matrix by showing one letter at a time.

Hub: Turn Off Pixels



This block turns off all the lights on the 5x5 Light Matrix.

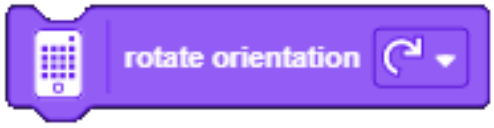
Hub: Set Pixel Brightness To (Value) %

This block sets the brightness of the 5x5 Light Matrix for the next block using the 5x5 Light Matrix, such as the *Turn on 5x5 Matrix* Block. If the brightness has not been specified, the default value is 100%.

Hub: Set Pixel At (X▼), (Y▼) To (Value) %

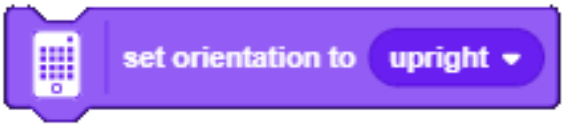
This block sets the brightness of individual pixels on the 5x5 Light Matrix. Only the specified pixel is updated. The rest of the 5x5 Light Matrix remains unchanged.

Hub: Rotate Orientation (Direction▼)



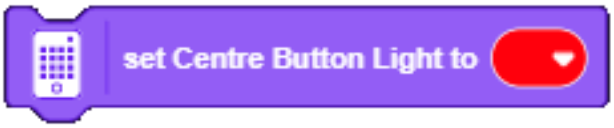
This block rotates the orientation either clockwise or counterclockwise of what is displayed on the 5x5 Light matrix.

Hub: Set Orientation To (Direction▼)



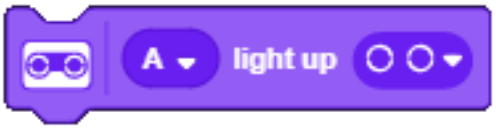
This block sets the orientation of what is displayed on the 5x5 Light matrix. The orientation can be set to upright, upside down, left, or right. The default orientation is upright.

Hub: Set Centre Button Light To (Colour▼)



This block sets the colour of the Centre Button light.

Distance Sensor: (Hub Port▼) Light Up (Lights Pattern▼)



This block turns on the light segments on the Distance Sensor when one is connected.

Sound Blocks

Sound Blocks enable you to play sounds from your device or Hub.

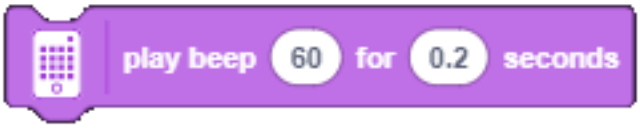
App: Play Sound (Name▼) Until Done

This block plays a selected sound on your device and pauses the programming stack until the sound is finished. Sounds can be added to the project using the Add Sound Button.

App: Start Sound (Name▼)

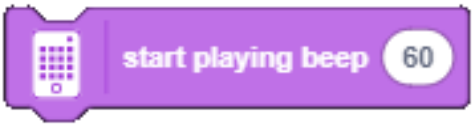
This block starts playing a selected sound on your device and immediately plays the next block in the programming stack. Sounds can be added to the project using the Add Sound Button.

Hub: Play Beep (Tone Value) For (Value) Seconds



This block plays a beep tone on your Hub for a specified number of seconds.

Hub: Start Playing Beep (Tone Value)

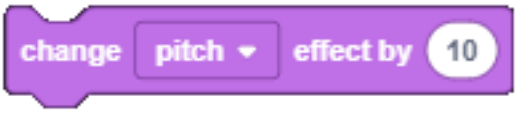


This block plays a beep tone on your Hub. It will keep playing until something in the program stops it.

App & Hub: Stop All Sounds

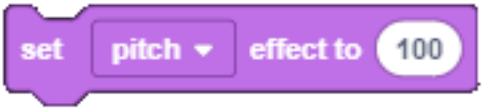
This block stops all sounds that are currently playing (i.e., beeps and sound files).

App: Change (Pitch Or Pan▼) Effect By (Value)



This block changes the pitch or pan left/right effect of the sounds played on the device.
The pan effect changes which speaker the sound is coming out of, with -100 being only the left speaker, 0 being normal, and 100 being only the right speaker.

App: Set (Pitch Or Pan▼) Effect To (Value)



This block sets the pitch or pan left/right effect of the sounds played on the device. The pan effect changes which speaker the sound is coming out of, with -100 being only the left speaker, 0 being normal, and 100 being only the right speaker. "

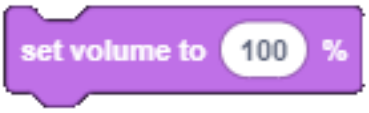
App: Clear Sound Effects

This block sets both the pitch and pan left/right sound effect back to normal.

App: Change Volume By (Value)

This block changes the volume of the sound currently being played by a specified increment from the volume at which it is currently playing. The default volume is 100%.

App: Set Volume To (Value)



This block sets the volume of the sound. The default volume is 100%.

App: =Volume

This block reports the current sound volume when activated. Tick the box to turn it on. Drag it into an Operator block to use it.

Event Blocks

Event Blocks are comprised entirely of Hat Blocks, meaning that they are always the first block in a programming stack and other blocks can only be attached under them. Hat Blocks are necessary to start a programming stack and will be triggered when a specified event occurs.

When Program Starts

This block plays all the blocks attached to it, sequentially from top to bottom, when the program starts. The program can be started by clicking the Play Button or, when not in Streaming Mode, by pressing the Centre Button on the Hub.

Colour Sensor: (Hub Port▾) When Colour Is (Colour▾)

This block plays all the blocks attached to it when the Colour Sensor detects a specified colour. The detectable colours are:

(0) Black
(1) Violet
(3) Blue
(4) Light Blue
(5) Green
(7) Yellow
(9) Red
(10) White
(-1) no colour

This block will only trigger when it detects the specified colour. This means that the block will not re-trigger if the detected colour remains unchanged.

Distance Sensor: (Hub Port▾) When (Closer/Further Than▾) (Value) (Units)

This block plays all the blocks attached to it when the Distance Sensor detects that an object is closer than or further than the specified distance. This block will only trigger at the specified event. This means that the block will not re-trigger if the distance remains unchanged.

Hub: When (Hub Orientation▼) Is Up

This block plays all the blocks attached to it when the Hub is placed in the specified orientation. The possible orientations are:

(1) Front
(2) Back
(3) Top
(4) Bottom
(5) Left side
(6) Right side

This block will only trigger at the specified event. This means that the block will not re-trigger if the orientation of the Hub remains unchanged.

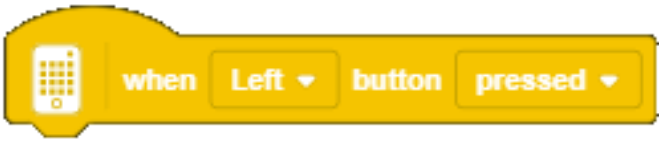
Hub: When (Shaken/Tapped/Falling▼)

This block plays all the blocks attached to it when the Hub is:

(1) Shaking
(2) Tapped
(3) Falling

This block will only trigger at the start of the specified event. This means that the block will not re-trigger if the movement of the Hub remains unchanged.

Hub: When (Left/Right▼) Button (Pressed/Released▼)



This block plays all the blocks attached to it when the Left or Right Buttons are pressed or released. This block will only trigger at the specified event. This means that the block will not re-trigger as long as the state of the Buttons remains unchanged.

When <Condition>

This block plays all the blocks attached to it when a certain condition is true. This block will only trigger at the event of the specified condition becoming true. This means that the block will not re-trigger if the condition remains true.

When I Receive (Message▼)

This block plays all the blocks attached to it when a specified message is broadcasted by the *Broadcast Message* Block or the *Broadcast Message and Wait* Block.

Broadcast (Message▼)

This block broadcasts a specified message. All the *When I Receive Message* Hat Blocks that have been set to the specified message will play. After the message has been sent, the next block in the programming stack will play.

Broadcast (Message▼) and Wait

This block broadcasts a specified message. All the *When I Receive Message* Hat Blocks that have been set to the specified message will play. After the message has been sent, the block waits until all the programming stacks with the specified message are finished before it proceeds to the next block in the stack.

When Timer > (Value)

This block plays all the blocks attached to it when the timer exceeds a specified value. This block will only trigger at the event of the timer exceeding the specified value. This means that the block will not re-trigger as long as the timer has already exceeded the specified value.

When (Key Name▼) Key Pressed

This block plays all the blocks that are attached to it when the selected key is pressed down.

Control Blocks

The *Control Blocks* category contains all the blocks that can modify the linear flow of block execution, such as “wait for" structures, loops, and conditions.

Wait (Value) Seconds

This block pauses the programming stack for a specified number of seconds — it supports whole numbers and decimals.

Wait Until <Condition>

This block pauses the programming stack until the specified boolean condition is true.

Repeat (Value) [Block Stack]

All the blocks held inside this block will loop a specified number of times before allowing the programming stack to continue.

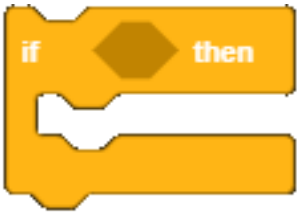
Forever [Block Stack]

All the blocks held inside this block will loop forever. The only way to stop the loop is to interrupt the program by pressing the Stop Button, or by using the Stop All Block.

Repeat Until <Condition> [Block Stack]

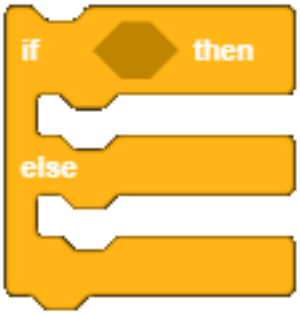
All the blocks held inside this block will loop until the specified boolean condition is true. When the specified condition becomes true, blocks beneath it will play.

If <Condition> Then [Block Stack]



This block will check whether the specified boolean condition is true. If the condition is true, all the blocks held inside it will play. If the condition is false, the blocks will be ignored.

If <Condition> Then [Block Stack] Else [Block Stack]



This block will check whether the specified boolean condition is true. If the condition is true, the blocks held inside the first space will play and the stack will continue. If the condition is false, the blocks inside the second space will play.

Stop Other Stacks

This block stops all programming stacks except its own.

Stop (Stacks▼)

This block stops all programming stacks that are currently running, or only its own programming stack.

Sensor Blocks

Sensor Blocks receive information from the sensors (e.g., Colour Sensor, Distance Sensor, Force Sensor, Gyro Sensor).

Colour Sensor: =(Hub Port▼) Is Colour (Colour▼)?

This block returns "true" when the Colour Sensor detects the specified colour. The detectable colours are:

(0) Black
(1) Violet
(3) Blue
(4) Light Blue
(5) Green
(7) Yellow
(9) Red
(10) White
(-1) No colour

Colour Sensor: =(Hub Port▼) Colour

This block returns the current value of the colour detected by the Colour Sensor. The detectable colours are:

(0) Black
(1) Violet
(3) Blue
(4) Light Blue
(5) Green
(7) Yellow
(9) Red
(10) White
(-1) No colour

Colour Sensor: =(Hub Port▼) Reflection (Compare▼) (Value) %?

This block returns “true" when the light reflected back to the Colour Sensor is greater than, equal to, or less than the specified percentage.

Colour Sensor: =(Hub Port▼) Reflected Light

This block reports the current value of light that is being reflected back to the Colour Sensor.

Distance Sensor: =(Hub Port▼) Is (Closer/Further Than▼) (Value) (Units▼)?

This block returns “true" when the Distance Sensor detects that something is closer than (<), further than (>), or exactly at (=) a distance specified in centimetres, inches, or as a percentage.

Distance Sensor: =(Hub Port▼) Distance In (Units▼)

This block reports the current distance the Distance Sensor is detecting in centimetres, inches, or as a percentage. The sensor's range is 0-200 centimetres or 0-78.74 inches.

Hub: =Is (Hub Orientation▼) Up?

This block returns “true" when the Hub is placed in the specified orientation. The possible orientations are:

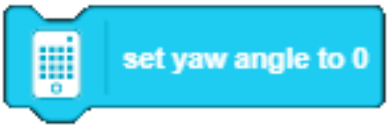
(1) Front
(2) Back
(3) Top
(4) Bottom
(5) Left side
(6) Right side

Hub: =(Pitch/Roll/Yaw▼) Angle

This block reports the Hub's pitch, roll, or yaw angle. *Pitch*, *roll*, and *yaw* are terms commonly used to describe an airplane's movement through air, but they can apply to any object rotating in all three dimensions. When looking at an airplane:

- The *pitch angle* refers to the airplane's nose going up or down.
- The *roll angle* refers to the airplane's wings going up or down.
- The *yaw angle* refers to the direction of the airplane compared to the ground.

Hub: Set Yaw Angle to 0



This block sets the yaw angle of the Hub to"0." By default, the yaw angle will be"0" in the direction in which the Hub is facing when the program starts.

Hub: =Is (Left/Right▾) Button (Pressed/Released▾)?

This block returns “true" if the Left or Right Button is pressed or released.

App: =Key (Key Name▾) Pressed?

This block returns "true" when the selected key is pressed down.

Hub: =Timer

This block reports the time, in seconds, since the program started. The timer restarts every time the program restarts. Use the Reset Timer Block to manually restart the timer.

Hub: Reset Timer

This block resets the timer.

Hub: =Gesture

This block reports the current gesture performed on the Hub. The Hub can detect the following gestures:

(1) shaking
(2) tapped
(3) falling

Hub: =Orientation

This block reports the current orientation of the Hub. The Hub can detect the following orientations:

(1) Front
(2) Back
(3) Top
(4) Bottom
(5) Left side
(6) Right side

Operator Blocks

Operator Blocks perform all the mathematical operations that can be done using numerical values.

=Pick Random (Start Value) To (End Value)

This block picks a random number within the specified range, including both endpoints. You can specify whole numbers or decimals as the minimum and maximum values. If one of the numbers is a decimal, it will report a decimal.

=(Value1) Plus (Value2)

This block adds two values and returns the result.

=(Value1) Minus (Value2)

This block subtracts the second value from the first value and returns the result.

=(Value1) Multiplied By (Value2)

This block multiplies two values and returns the result.

=(Value2) Divided By (Value1)

This block divides the second value by the first value and returns the result.

=(Value1) Greater Than (Value2)

This block checks whether the first value is greater than the second value. If it is greater, it will return “true”. If not, it will return “false".

=(Value1) Less Than (Value2)

This block checks whether the first value is less than the second value. If it is less, it will return “true". If not, it will return “false".

=(Value1) Equal To (Value2)

This block checks whether the first value is equal to the second value. If it is equal, it will return “true". If not, it will return “false".

=<Boolean Block1> And <Boolean Block2>

This block joins two Boolean Blocks with an “AND" condition.

=<Boolean Block1> Or <Boolean Block2>

This block joins two Boolean Blocks with an “OR" condition.

=Not <Boolean Block>

This block inverts the boolean value of the condition inside it.

=Is (Value1) In Between (Value2) And (Value3)?

This block checks whether the first value specified falls between the second and third specified values, including both endpoints.

=Join (String1) (String2)

This block joins two text values and returns the result (e.g., if “hello" and “world" were input into the block, it would return "helloworld").

=Letter (Value) Of (String)

This block returns the character that occupies the specified position of the given string. As an example, "letter 1 of LEGO" will return "L."

=Length Of (String)

This block returns the number of characters contained in the given string. For example, if the input is “LEGO," the block returns "4."

=(String) Contains (Character)?

This block returns “true" if the specified character is contained in the specified string.

=(Value1) Mod (Value2)

This block returns the remainder when the first value is divided by the second value (e.g., when *10* is the first input and *3* is the second, the block will report *1*; *10* divided by *3* gives a remainder of *1*). Negative numbers behave a little differently because a remainder must always be positive (e.g., *-10* mod *3* equals *2*, not *-1* because you have to multiply *3* by *-4* to have any remainder at all).

=Round (Value)

This block rounds the given number to the nearest integer. It follows the standard rules of rounding (i.e., decimals of .5 or higher are rounded up, whereas decimals of less than .5 are rounded down).

=(Math Function) Of (Value)

This block performs the specified math function on a given number and reports the result.

Controller Commands

These blocks are used by the user-designable Remote Control.

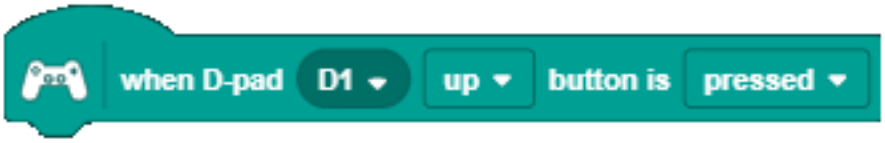
App RC: When Joystick (J Name▼) Is (Position▼)

This block plays all the blocks attached to it when the Joystick is in the specified state. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the Joystick remains unchanged.

App RC: =Joystick (J Name▼) (Axis▼)

This block reports the current position of the Joystick along the specified axis. The position will be "0" when the Joystick is released.

App RC: When D-Pad (D Name▼) (B Name▼) Button Is (Pressed/Released▼)



This block plays all the blocks attached to it when the specified Button on the D-pad is pressed or released. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the D-pad remains unchanged.

App RC: =Is D-Pad (D Name▼) (B Name▼) Button (Pressed/Released▼)?

This block returns "true" if the specified Button on the D-pad is pressed or released.

App RC: When Slider (S Name▼) Is (Position▼)

This block plays all the blocks attached to it when the Slider is in the specified state. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the Slider remains unchanged.

App RC: =Slider (S Name▼)

This block reports the current position of the Slider. The position will be "0" when the Slider is released.

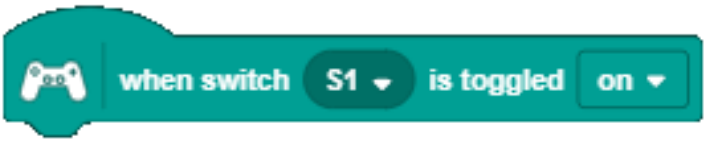
App RC: When Button (B Name▼) Is (Pressed/Released▼)

This block plays all the blocks attached to it when the Button is pressed or released. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the Button remains unchanged.

App RC: =Is Button (B Name▼) (Pressed/Released▼)?

This block returns "true" if the Button is in the specified state.

App RC: When Switch (S Name▼) Is Toggled (On/Off▼)



This block plays all the blocks attached to it when the Switch is toggled on or off. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the Switch remains unchanged.

App RC: =Is Switch (S Name▼) (On/Off▼)?

This block returns "true" if the Switch is in the specified state.

Variable Blocks

The *Variable Blocks* category contains all the blocks linked to variables, lists (arrays), and My Blocks.

Make A Variable

Makes a new Variable block.

=(MyVariable)

This block reports the value of a variable. Whenever a variable is created, a version of this block appears with the variable's given name on it.

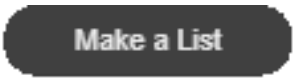
Set (Variable▼) To(Value)

This block sets the specified variable to the given value. The variable can be either a string or a number.

Change (Variable▼) By (Value)

This block changes the specified variable by a given value. The change is from the specified amount from the value currently stored in the variable. For example, if my variable contains the value *4*, using the Change Variable By 3 Block would make the value change to *7*. Also, if the variable is a text string (not a number), the value of the variable is set to the quantity the variable was to be changed by. For example, if “my variable” contains "LEGO", using the block shown above will change the value to"1."

Make A List

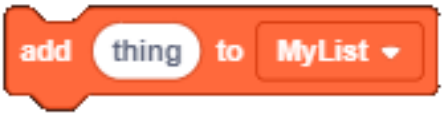


Makes a new List block

=(MyList)

This block reports, as a string, the items contained in a list. Whenever a list is created, a version of this block appears with the list's name on it.

Add (Item) To (List▾)

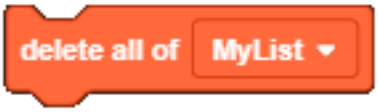


This block adds the specified item to the end of the specified list.

Delete (Index) Of (List▾)

This block deletes the item that is currently occupying the specified position in the specified list.

Delete All Of (List▾)

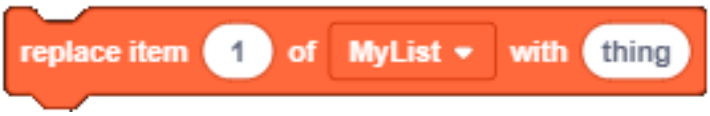


This block deletes all the items in the specified list.

Insert (Item) At (Index) Of (List▾)

This block inserts a specified item at a specific position in the specified list.

Replace Item (Index) Of (List▾) With (Item)



This block replaces the item at the specified position with a specified value.

=Item (Index) Of (List▾)

This block returns the value that occupies the specified position in a specified list.

=Item # Of (Item) In (List▾)

This block returns the number of the position in a list where an item first appears. If the item is not contained in the specified list, it reports"0."

=Length Of (List▾)

This block returns the number of items contained in the specified list.

=(List▾) Contains (Item)?

This block returns “true" if the list contains the specified value in any position. The specified value must be an exact match for the value contained in the list. If none of the values in the list are equal to the specified value, it returns “false."

My Block

This is your block! It will play whatever you have attached to the Define Block.

Make A Block

This block allows you to create your own block. A *My Block* is the group of blocks that is attached to the Defined Block.

MyBlock

Whenever a block is created, a version of this block appears with the block's given name on it.

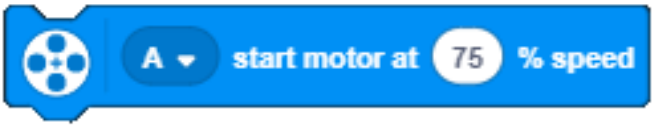
More Motor Blocks

These blocks extend functionality of the Motor Blocks to give you even more programming possibilities!

Motor: (Hub Port▾) Run For (Value) (Unit▾) At (Value) % Speed

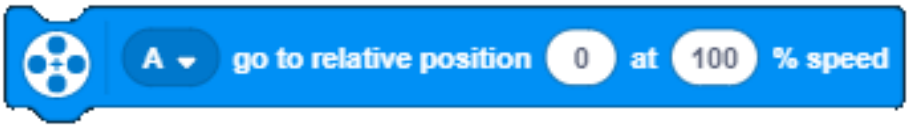
This block will run one or more motors clockwise or counterclockwise for a specified number of rotations, seconds, or degrees at a speed specified as a percentage or in revolutions per minute. “Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: (Hub Port▾) Start Motor At (Value) % Speed



This block will run one or more motors clockwise or counterclockwise forever at a speed specified as a percentage or in revolutions per minute. “Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: (Hub Port▼) Go To Relative Position (Value) At (Value) % Speed



This block runs one or more motors to a relative position at a specified speed. Unlike the absolute position used in the *Go To Position* Block, the relative position has no range limit and can be reset with the *Set Relative Motor Position to 0* Block.
'Stall detection' is enabled by default See the *Turn Stall Detection On/Off Block* for more information.

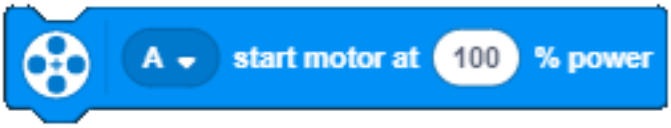
Motor: (Hub Port▼) Set Relative Position To (Value)

This block sets the relative position of one or more motors to a specified value. Use a value of '0' to reset the relative position.

Motor: =(Hub Port▼) Degrees Counted

This block returns the number of degrees that the specified motor has turned since the program started or was reset by the *Set Relative Motor Position to 0* Block.

Motor: (Hub Port▼) Start Motor At (Value) % Power

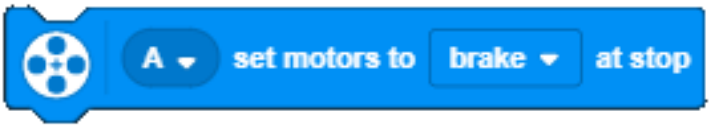


This block will run one or more motors at a specified percentage of power forever. When running a motor according to speed, the power of the motor is regulated in order to maintain the specified speed.
“Stall detection" is enabled by default. See the *Turn Stall Detection On/Off Block* for more information.

Motor: =(Hub Port▼) Power

This block returns, as a percentage, the power level currently being used on the specified motor.

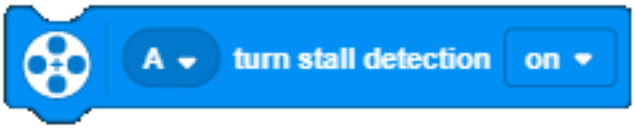
Motor: (Hub Port▼) Set Motors To (Brake/Hold/Float▼) At Stop



This block specifies how the motor will stop when using a Motor Block with a specified duration, or the Stop Motor Block. The motor can stop in three different ways:

- *Brake*: the default method in which the motor uses power to brake when stopping and applies friction to the motor afterward
- *Hold position*: the motor uses power to brake and actively moves the motor back to the position in which it stopped, if it is forced away from it
- *Float*: the power to the motor is cut when stopping.

Motor: (Hub Port▼) Turn Stall Detection (On/Off▼)



This block will disable stall detection on the Motor and Movement Blocks. With stall detection turned off, the motors will keep trying to complete their command even when they are physically stopped, which will halt the rest of the program. By default, all motor and movement commands will detect when a motor has stalled. A *motor stall* means that something has physically prevented the motor from completing its command. When stalling occurs, the program will continue to the next block in the programming stack.

Motor: (Hub Port▼) Was The Motor Interrupted?

This block returns “true" if the selected motor got interrupted. *Got interrupted* means that the last Motor Block with a specified duration did not complete its full duration because a programming block has interrupted it.

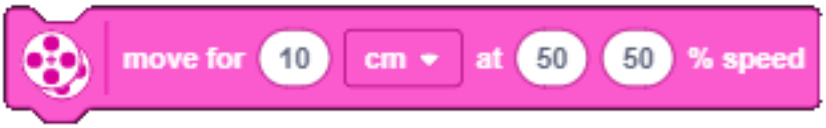
Motor: (Hub Port▼) Set Acceleration To (Rate▼)

This block sets the acceleration and deceleration of one or more motors.

More Movement Blocks

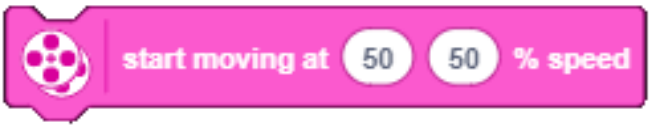
These blocks extend the functionality of the Movement Blocks to give your even more programming possibilities for your Driving Base.

Motor Pair: Move For (Value) (Units▼) At (L Value) (R Value) % Speed



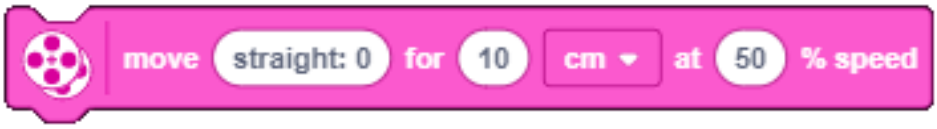
This block moves a Driving Base for a specified number of centimetres, inches, seconds, degrees, or rotations at a specified speed for each motor. The first specified speed value sets the speed of the left motor, and the second specified value sets the speed of the right motor.

Motor Pair: Start Moving At (L Value) (R Value) % Speed



This block starts moving a Driving Base forever at the speed that has been specified for each motor. The first specified speed value sets the speed of the left motor, and the second specified value sets the speed of the right motor.

Motor Pair: Move (Value Steering) For (Value) (Units▼) At (Value) % Speed

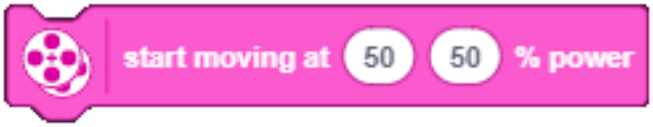


This block moves a Driving Base for a specified duration (i.e., centimetres, inches, seconds, degrees, or rotations) at a specified speed for each motor, while steering left (-100), right (100), or straight (0).

Motor Pair: Start Moving (Value Steering) At (Value) % Speed

This block starts moving a Driving Base at a specified speed for each motor while steering left (-100), right (100), or straight (0) forever.

Motor Pair: Start Moving At (L Value) (R Value) % Power



This block starts moving a Driving Base forever at the power that has been specified for each motor. The first specified value sets the power of the left motor, and the second specified value sets the power of the right motor.

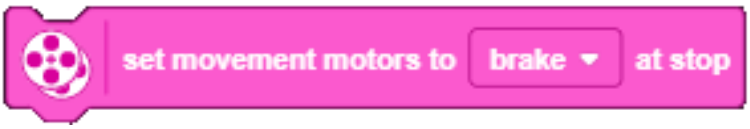
- Running the motors with power will, unlike running the motors with speed, not regulate the speed of the motors. Use power for robots that should be regulated on other inputs than speed, such as a line-following robot or a self-balancing-robot.

Motor Pair: Start Moving (Value Steering) At (Value) % Power

This block starts moving a Driving Base forward forever using a steering value and a specified percentage of power.

- Higher steering values (i.e., +99 and -99) will make the arc path of the Driving Base sharper. Use a value of"0" to drive straight forward. Using the values 100 and -100 will make the Driving Base pivot on itself.
- Running the motors with power will, unlike running the motors with speed, not regulate the speed of the motors. Use power for robots that should be regulated on other inputs than speed, such as a line-following robot or a self-balancing-robot.

Motor Pair: Set Movement Motors To (Brake/Hold/Float▾) At Stop



This block specifies how motors will stop when using a Movement Block with a specified duration, or the Stop Moving Block. The motor can stop in three different ways:

1. *Brake*: the default method in which the motor uses power to brake when stopping and applies friction to the motor afterward
2. *Hold position*: the motor uses power to brake and actively moves the motor back to the position in which it stopped, if it is forced away from it
3. *Float*: the power to the motor is cut when stopping

Motor Pair: =Did Movement Get Interrupted?

This block returns “true" if the movement got interrupted. *Got interrupted* means that the last Motor Block with a specified duration did not complete its full duration because a programming block has interrupted it.

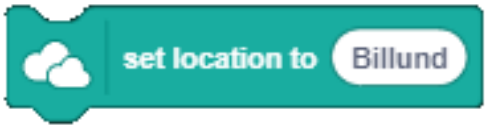
Motor Pair: Set Movement Acceleration To (Rate▾)

This block sets the acceleration and deceleration of a Driving Base

Weather Blocks

The available Weather Blocks are based on forecasts, not current values. Students can play around with forecasts up to 240 hours ahead of the current time. Weather Blocks will only work when you are online because they fetch real time data.

App Weather: Set Location To (String)

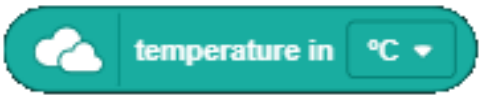


This block sets the location for which weather forecast data will be retrieved.

App Weather: =Location

This block reports the location currently set by the Set Forecast Location Block.

App Weather: =Temperature In (°C/°F▼)

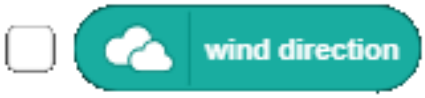


This block reports the temperature at the set forecast location in either Celsius or Fahrenheit.

App Weather: =Wind Speed In (Units▼)

This block reports the wind speed at the set forecast location in meters per second or miles per hour.

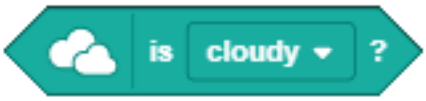
App Weather: =Wind Direction



This block reports the wind direction at the set forecast location. The direction is given as a string:

"N" for north
"NE" for northeast
"E" for east
"SE" for southeast
"S" for south
"SW" for southwest
"W" for west
"NW" for northwest

App Weather: =Is (Weather Condition▼)?



This block returns “true" if there is a match for the specified weather condition at the set forecast location.

App Weather: =Precipitation In (Units ▼)

This block returns, in millimetres or inches, the amount of rain, snow, or other water condensation that is expected to fall at the set forecast location at a given time.

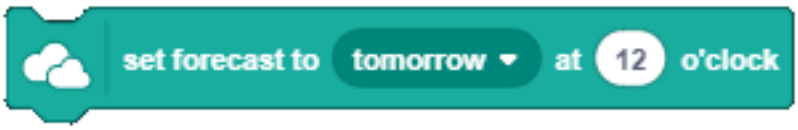
App Weather: =Air Pressure In hPa

This block returns the atmospheric pressure, in hectopascal (hPa), at the set forecast location.

App Weather: Set Forecast To Now

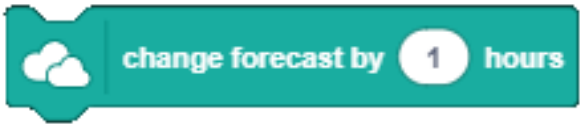
This block sets the forecast time of the weather data to now.

App Weather: Set Forecast To (Day ▼) At (Value) O’Clock



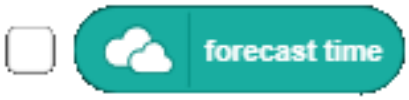
This block sets the weather forecast from 1 to 9 days into the future at a specified hour based on a 24-hour clock.

App Weather: Change Forecast By (Value) Hours



This block changes the forecast time by a specified number of hours from the current set value. The time is based on a 24-hour clock.

App Weather: =Forecast Time



This block reports the number of hours that the forecast has been shifted into the future from the current time. For example, if the current time is 10: 00 and the forecast time has been changed to 16:00 today, this block will return a value of "6".

Music Blocks

These blocks can be used to create musical sounds. Combine them all to create your own symphony!

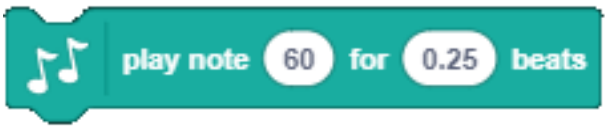
App Music: Play Drum (Name▼) For (Value) Beats

This block plays the specified drum for the specified amount of time, measured in beats.

App Music: Rest For (Value) Beats

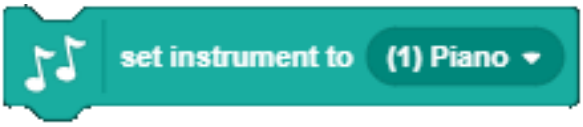
This block “plays" a rest for the specified amount of time, measured in beats.

App Music: Play Note (Tone) For (Value) Beats



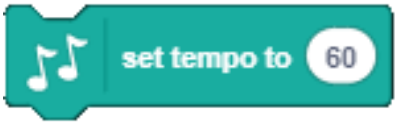
This block plays the specified note for the specified amount of time, measured in beats.

App Music: Set Instrument To (Name▼)



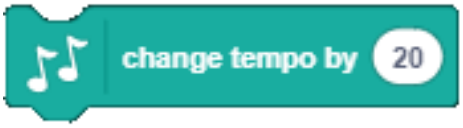
This block sets the instrument used in the Play Note Block.

App Music: Set Tempo To (Value)



This block sets the tempo of the beat. The tempo is represented as *beats per minute (bpm)*. *60 bpm* means that one beat will be played every second.

App Music: Change Tempo By (Value)



This block changes the tempo of the beat that is currently playing. The tempo is represented as *beats per minute (bpm)*. *60 bpm* means that one beat will be played every second.

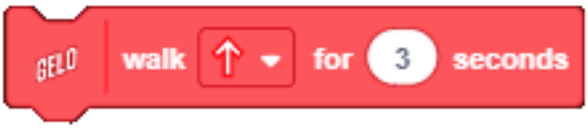
App Music: =Tempo

This block reports the tempo that is currently set.

Gelo: Commands

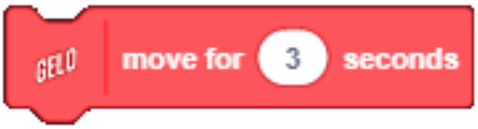
These commands apply to Gelo robots.

Gelo: Walk (Direction▼) For (Value) Seconds



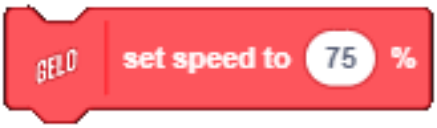
This block makes Gelo walk in the specified direction for the specified number of seconds. Gelo's speed is set by the *Gelo Set Speed* block. The default speed is 75%.

Gelo: Move For (Value) Seconds



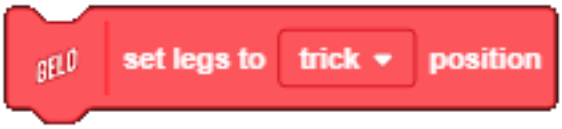
This block makes Gelo move for the specified number of seconds. Gelo's speed and direction are set by the *Gelo Walk for Seconds* and *Gelo Set Speed* blocks. The default speed is 75%.

Gelo: Set Speed To (Value) %



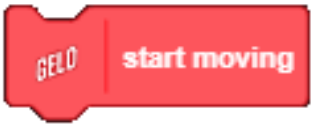
This block sets the speed at which Gelo moves. The speed range is -100 to 100. Negative values will move Gelo backward. The default speed is 75%.

Gelo: Set Legs To (P Name▼) Position



This block sets Gelo's legs to the specified position. Play this block before running the *Gelo Move for Seconds* and *Gelo Start Moving* Blocks to set the way Gelo moves.

Gelo: Start Moving



This block starts moving Gelo forever. Use the *Gelo Stop* Block to stop Gelo. The way Gelo moves is set by the *Gelo Set Legs to Position* Block. Gelo's speed and direction are set by the *Gelo Set Speed* Block. The default speed is 75%.

Gelo: Stop

This block stops Gelo by turning off the motors.

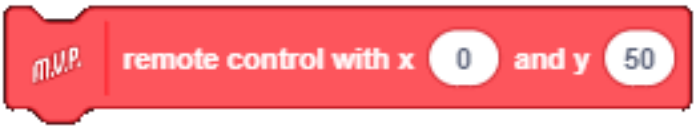
M.V.P.: Commands

These commands apply to M.V.P. robots.

M.V.P.: Calibrate

This block calibrates the M.V.P. steering motor on port A and runs it to the neutral position for driving straight.

M.V.P.: Remote Control With X (ValueX) And Y (ValueY)

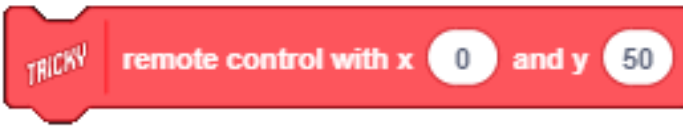


This block takes x and y values to determine how to move M.V.P. Use this block in a loop, and drop Reporter Blocks, such as the *Joystick* Block on the inputs to control M.V.P.

Tricky: Commands

These commands apply to Tricky robots.

Tricky: Remote Control With X (ValueX) And Y (ValueY)



This block takes x and y values to determine how to move Tricky. The block applies the following logic:

- if both the x and y values are small, stop moving
- if the y value is large, move forward or backward with "dampened" steering
- if the y value is small, turn in place using the x value

Use this block in a loop, and drop Reporter Blocks, such as the Joystick Block on the inputs to control Tricky.

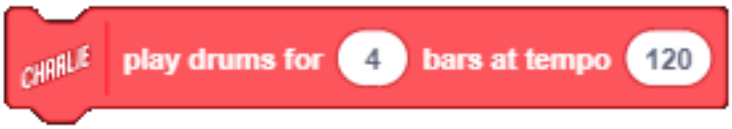
Charlie: Commands

These commands apply to Charlie robots.

Charlie: Calibrate

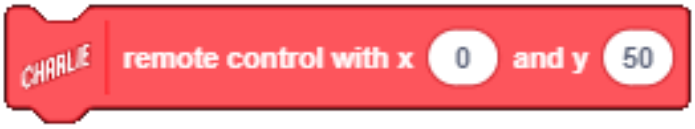
This block calibrates Charlie's arm motors on ports B and F and runs them to the neutral position.

Charlie: Play Drums For (Value) Bars At Tempo (Value)



This block runs Charlie's arm motors on ports B and F to play the drums for the specified number of bars at the specified tempo.

Charlie: Remote Control With X (ValueX) And Y (ValueY)



See Tricky Remote Control

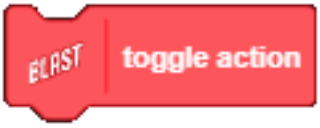
Blast: Commands

These commands apply to Blast robots.

Blast: Calibrate

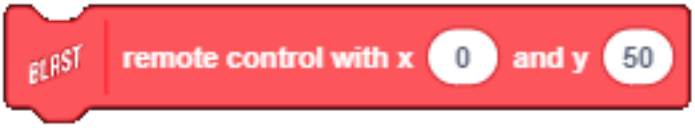
This block calibrates Blast's arms and head motor on port D and runs it to the neutral position.

Blast: Toggle Action



This block runs Blast's extension motor on port B. Each time this block is played it runs the motor in the opposite direction.

Blast: Remote Control With X (ValueX) And Y (ValueY)



See Tricky Remote Control

More Controller Commands

These blocks are used with these two Controllers:

➔ DualShock®4 Controller

➔ Xbox One® Controller

Behaviour and control functions vary slightly depending on which Controller is used.

DualShock®4 RC: When (S Name▾) Stick Is (Position▾)

Xbox One® RC: When (S Name▾) Stick Is (Position▾)

This block plays all the blocks attached to it when the selected stick is in the specified state. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the selected stick remains unchanged.

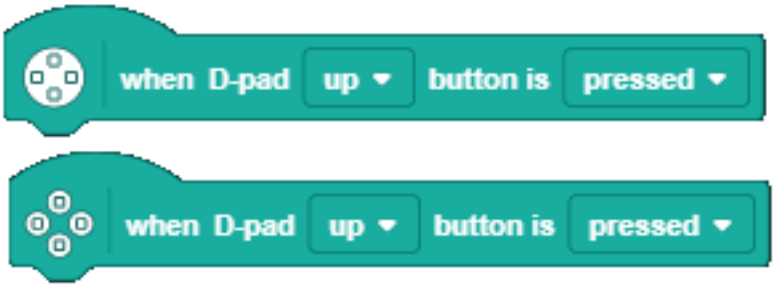
DualShock®4 RC: =(S Name▼) Stick (Axis▼)

Xbox One® RC: =(S Name▼) Stick (Axis▼)

This block reports the current position of the selected stick along the specified axis. The position will be "0" when the selected stick is released.

DualShock®4 RC: When D-Pad (B Name▼) Button Is (Pressed/Released▼)

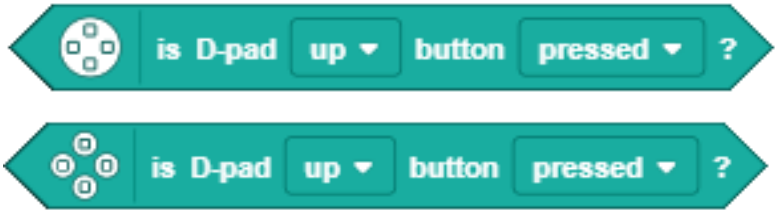
Xbox One® RC: When D-Pad (B Name▼) Button Is (Pressed/Released▼)



This block plays all the blocks attached to it when the specified Button on the D-pad is pressed or released. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the D-pad remains unchanged.

DualShock®4 RC: =Is D-Pad (B Name▼) Button (Pressed/Released▼)?

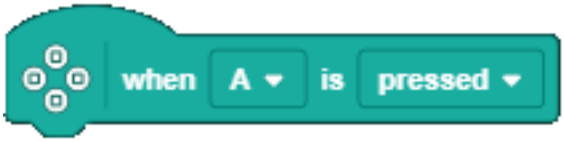
Xbox One® RC: =Is D-Pad (B Name▼) Button (Pressed/Released▼)?



This block returns "true" if the specified Button on the D-pad is pressed or released.

DualShock®4 RC: When (B Name▼) Is (Pressed/Released▼)

Xbox One® RC: When (B Name▼) Is (Pressed/Released▼)



This block plays all the blocks attached to it when the selected button is pressed or released. This block will only trigger at the specified event. This means that the block will not re-trigger if the state of the button remains unchanged.

DualShock®4 RC: =Is (B Name▼) (Pressed/Released▼)?

Xbox One® RC: =Is (B Name▼) (Pressed/Released▼)?

This block returns "true" if the selected button is in the specified state.

DualShock®4 RC: =(T Name▼) Pressure

Xbox One® RC: =(T Name▼) Pressure

This block reports the current pressure of the selected trigger. The pressure will be "0" when the selected trigger is released.

Index

(Character) 10
(Name▼) 10
(String) 10
(Value) 9

[Block Stack] 10

<Condition> 10

=(List▼) Contains (Item)? 34
=(Math Function) Of (Value) 29
=(MyList) 32
=(MyVariable) 32
=(String) Contains (Character)? 29
=(Value1) Equal To (Value2) 28
=(Value1) Greater Than (Value2) 28
=(Value1) Less Than (Value2) 28
=(Value1) Minus (Value2) 27
=(Value1) Mod (Value2) 29
=(Value1) Multiplied By (Value2) 27
=(Value1) Plus (Value2) 27
=(Value2) Divided By (Value1) 27
=<Boolean Block1> And <Boolean Block2> 28
=<Boolean Block1> Or <Boolean Block2> 28
=Is (Value1) In Between (Value2) And (Value3)? 28
=Item # Of (Item) In (List▼) 33
=Item (Index) Of (List▼) 33
=Join (String1) (String2) 29
=Length Of (List▼) 33
=Length Of (String) 29
=Letter (Value) Of (String) 29
=Not <Boolean Block> 28
=Pick Random (Start Value) To (End Value) 27
=Round (Value) 29

Add (Item) To (List▼) 33
App
  =Key (Key Name▼) Pressed? 26
  =Volume 17
  Change (Pitch Or Pan▼) Effect By (Value) 17
  Change Volume By (Value) 17
  Clear Sound Effects 17
  Play Sound (Name▼) Until Done 16
  Set (Pitch Or Pan▼) Effect To (Value) 17
  Set Volume To (Value) 17

Start Sound (Name▼) 16
App & Hub
Stop All Sounds 16
App Music
=Tempo 42
Change Tempo By (Value) 42
Play Drum (Name▼) For (Value) Beats 41
Play Note (Tone) For (Value) Beats 42
Rest For (Value) Beats 42
Set Instrument To (Name▼) 42
Set Tempo To (Value) 42
App Music: 9
App RC
=Is Button (B Name▼) (Pressed/Released▼)? 31
=Is D-Pad (D Name▼) (B Name▼) Button (Pressed/Released▼)? 30
=Joystick (J Name▼) (Axis▼) 30
=Slider (S Name▼) 31
Is Switch (S Name▼) (On/Off▼)? 31
When Button (B Name▼) Is (Pressed/Released▼) 31
When D-Pad (D Name▼) (B Name▼) Button Is (Pressed/Released▼) 30
When Joystick (J Name▼) Is (Position▼) 30
When Slider (S Name▼) Is (Position▼) 30
When Switch (S Name▼) Is Toggled (On/Off▼) 31
App RC: 9
App Weather
=Air Pressure In hPa 40
=Forecast Time 41
=Is (Weather Condition▼)? 40
=Location 39
=Precipitation In (Units ▼) 40
=Temperature In (°C/°F▼) 39
=Wind Direction 40
=Wind Speed In (Units▼) 40
Change Forecast By (Value) Hours 41
Set Forecast To (Day ▼) At (Value) O'Clock 41
Set Forecast To Now 41
Set Location To (String) 39
App Weather: 9
App: 9

Blast
Calibrate 46
Commands 46
Remote Control With X (ValueX) And Y (ValueY) 46
Toggle Action 46
Block Stack 8
Boolean Blocks 7
Broadcast (Message▼) 20
Broadcast (Message▼) and Wait 20

C Blocks 6
Cap Blocks 7
Change (Variable▼) By (Value) 32
Charlie
  Calibrate 45
  Commands 45
  Play Drums For (Value) Bars At Tempo (Value) 45
  Remote Control With X (ValueX) And Y (ValueY) 45
Colour Sensor
  (Hub Port▼) When Colour Is (Colour▼) 18
  =(Hub Port▼) Colour 24
  =(Hub Port▼) Is Colour (Colour▼)? 24
  =(Hub Port▼) Reflected Light 25
  =(Hub Port▼) Reflection (Compare▼) (Value) %? 24
Colour Sensor: 9
Component Names 9
Control Blocks 22
Controller Commands 30

Delete (Index) Of (List▼) 33
Delete All Of (List▼) 33
Distance Sensor
  (Hub Port▼) Light Up (Lights Pattern▼) 16
  (Hub Port▼) When (Closer/Further Than▼) (Value) (Units) 18
  =(Hub Port▼) Distance In (Units▼) 25
  =(Hub Port▼) Is (Closer/Further Than▼) (Value) (Units▼)? 25
Distance Sensor: 9
DualShock®4 RC
  =(S Name▼) Stick (Axis▼) 47
  =(T Name▼) Pressure 49
  =Is (B Name▼) (Pressed/Released▼)? 48
  =Is D-Pad (B Name▼) Button (Pressed/Released▼)? 48
  When (B Name▼) Is (Pressed/Released▼) 48
  When (S Name▼) Stick Is (Position▼) 47
  When D-Pad (B Name▼) Button Is (Pressed/Released▼) 47
DualShock®4 RC: 9

Event Blocks 18

Forever [Block Stack] 22

Gelo
  Commands 42
  Move For (Value) Seconds 43
  Set Legs To (P Name▼) Position 43
  Set Speed To (Value) % 43
  Start Moving 43
  Stop 43
  Walk (Direction▼) For (Value) Seconds 43

Hat Blocks 6
Hub
=(Pitch/Roll/Yaw▼) Angle 25
=Gesture 26
=Is (Hub Orientation▼) Up? 25
=Is (Left/Right▼) Button (Pressed/Released▼)? 26
=Orientation 27
=Timer 26
Play Animation (Name▼) Until Done 14
Play Beep (Tone Value) For (Value) Seconds 16
Reset Timer 26
Rotate Orientation (Direction▼) 15
Set Centre Button Light To (Colour▼) 15
Set Orientation To (Direction▼) 15
Set Pixel At (X▼), (Y▼) To (Value) % 15
Set Pixel Brightness To (Value) % 15
Set Yaw Angle to 0 26
Start Animation (Name▼) 14
Start Playing Beep (Tone Value) 16
Turn Off Pixels 15
Turn On (5x5 Pattern▼) 14
Turn On (5x5 Pattern▼) For (Value) Seconds 14
When (Hub Orientation▼) Is Up 19
When (Left/Right▼) Button (Pressed/Released▼) 19
When (Shaken/Tapped/Falling▼) 19
Write (String) 14
Hub: 9

If <Condition> Then [Block Stack] 23
If <Condition> Then [Block Stack] Else [Block Stack] 23
Insert (Item) At (Index) Of (List▼) 33

Light Blocks Category 14

M.V.P.
Calibrate 44
Commands 44
Remote Control With X (ValueX) And Y (ValueY) 44
Make A Block 34
Make A List 32
Make A Variable 32
More Controller Commands 46
More Motor Blocks 34
More Movement Blocks 37

Motor
(Hub Port▼) Go (Direction▼) To Position (Value) 10
(Hub Port▼) Go To Relative Position (Value) At (Value) % Speed 35
(Hub Port▼) Run (Direction▼) For (Value) (Units▼) 10
(Hub Port▼) Run For (Value) (Unit▼) At (Value) % Speed 34
(Hub Port▼) Set Acceleration To (Rate▼) 36
(Hub Port▼) Set Motors To (Brake/Hold/Float▼) At Stop 36
(Hub Port▼) Set Relative Position To (Value) 35
(Hub Port▼) Set Speed To (Value) % 11
(Hub Port▼) Start Motor (Direction▼) 11
(Hub Port▼) Start Motor At (Value) % Power 35
(Hub Port▼) Start Motor At (Value) % Speed 34
(Hub Port▼) Stop Motor 11
(Hub Port▼) Turn Stall Detection (On/Off▼) 36
(Hub Port▼) Was The Motor Interrupted? 36
=(Hub Port▼) Degrees Counted 35
=(Hub Port▼) Power 35
=(Hub Port▼) Position 11
=(Hub Port▼) Speed 11
Motor Blocks Category 10
Motor Pair
=Did Movement Get Interrupted? 39
Move (Direction▼) For (Value) (Units▼) 12
Move (Value Steering) For (Value) (Units▼) 12
Move (Value Steering) For (Value) (Units▼) At (Value) % Speed 37
Move For (Value) (Units▼) At (L Value) (R Value) % Speed 37
Set 1 Motor Rotation To (Value) (Units▼) Moved 13
Set Movement Acceleration To (Rate▼) 39
Set Movement Motors To (Brake/Hold/Float▼) At Stop 38
Set Movement Motors To (Hub Ports▼) 13
Set Movement Speed To (Value) % 12
Start Moving (Value Steering) 12
Start Moving (Value Steering) At (Value) % Power 38
Start Moving (Value Steering) At (Value) % Speed 37
Start Moving At (L Value) (R Value) % Power 38
Start Moving At (L Value) (R Value) % Speed 37
Stop Moving 12
Motor Pair: 9
Motor: 9
Movement Blocks 11
Music Blocks 41
My Block 34
MyBlock 34

Operator Blocks 27

Parameters 9

Repeat (Value) [Block Stack] 22
Repeat Until <Condition> [Block Stack] 22

Replace Item (Index) Of (List▼) With (Item) 33
Reporter Blocks 7

Sensor Blocks 24
Set (Variable▼) To(Value) 32
Sound Blocks 16
Stack Blocks 6
Stop (Stacks▼) 23
Stop Other Stacks 23

Tricky
  Commands 45
  Remote Control With X (ValueX) And Y (ValueY) 45

Variable Blocks 32

Wait (Value) Seconds 22
Wait Until <Condition> 22
Weather Blocks 39
When (Key Name▼) Key Pressed 21
When <Condition> 20
When I Receive (Message▼) 20
When Program Starts 18
When Timer > (Value) 20

Xbox One® RC
  =(S Name▼) Stick (Axis▼) 47
  =(T Name▼) Pressure 49
  =Is (B Name▼) (Pressed/Released▼)? 48
  =Is D-Pad (B Name▼) Button (Pressed/Released▼)? 48
  When (B Name▼) Is (Pressed/Released▼) 48
  When (S Name▼) Stick Is (Position▼) 47
  When D-Pad (B Name▼) Button Is (Pressed/Released▼) 47
Xbox One® RC: 9